UNIVERSITÀ DEGLI STUDI DI UDINE
hic sunt futura

DIPARTIMENTO DI SCIENZE MATEMATICHE, INFORMATICHE E FISICHE

PROGETTO DI LINGUAGGI E COMPILATORI

Parte 2 - Parziale

Francesco Bortuzzo **157430**
Leonardo Danelutti **158786**
Lorenzo Della Giustina **158854**

Anno Accademico 2024/2025

Indice

1. Scelte di design 1
1.a. Modifiche alla sintassi concreta 1
1.b. Risoluzione conflitti 1
2. Cose da controllare durante l'analisi semantica statica 2
3. Definizione del linguaggio 2
3.a. Tipi 3
3.a.I. Type system 3
3.b. Variabili 3
3.c. Costanti 3
3.d. Loop 3
3.e. Condizionali 4
3.f. Routine 4
3.g. Commenti 4
4. TODO 4
4.a. separatori 4
5. TODO: 5

1. Scelte di design

- Le dichiarazioni di variabili contengono obbligatoriamente l'assegnamento (motivazione: vedi pagina 3 punto 2 `implementazione-compilatori-imperativi.pdf`).
- Le costanti possono avere solo tipi primitivi (visto che le costanti non devono allocare spazio in memoria) (FIXME: questa è ovvia la toglierei).
- La stringa è considerata un tipo primitivo (vedi pagina 3 punto 3 `implementazione-compilatori-imperativi.pdf`).
- I due tipi di iterazione indeterminata sono derivabili dall'implementazione del loop come previsto dalla sintassi concreta di Blue.
- `fn` è syntax sugar di `fn()`.

1.a. Modifiche alla sintassi concreta

- Dalla sintassi concreta sono state tolte classi, pre post condizioni, etc. perchè non coerenti con quanto richiesto (FIXME: questa è ovvia la toglierei).
- Nelle function call no punto (ovviamente perchè non abbiamo le classi).
- Blue supporta solo commenti single line (con «–» o «==»). I commenti «==» sono documentazione per classi/funzioni ed è stata tolta dalla nostra implementazione perchè non richiesti. Inoltre sono stati aggiunti i commenti multi-line «».
- Aggiunta keyword `return` nelle funzioni (al posto di avere i nomi delle variabili di ritorno che avrebbero richiesto di avere dichiarazioni senza assegnamento).

1.b. Risoluzione conflitti

- Il conflitto state 105 LExp vs RExp non è un problema. Essendo le LExp sottoinsieme delle RExp basta scegliere sempre la Shift.
- Le dichiarazioni di funzioni, nell'ambiente più esterno, devono essere precedute dalla keyword `routines` similmente a quanto succede in Blue per le dichiarazioni delle routines nelle classi. Per evitare ambiguità tipo:

```
var
  a: Integer := b
main ...
```

  dove main potrebbe essere sia un nome di funzione che un nome di una variabile.
- Aggiunto `;` alla fine degli Statement (a parte If e Loop) e delle dichiarazioni di costanti e variabili per evitare ambiguità tipo:

```

```

```
var
  a: Integer := b
( ...
```

dove con un lookahead di 1 non è chiaro se «b (« è l'inizio di una chiamata di funzione oppure «(« è l'inizio di una LExp. Oppure ambiguità tipo:

```
var
  a: Integer := b
* ...
```

dove «*» potrebbe essere sia un deref che una moltiplicazione. Per lo stesso motivo è stato aggiunto un «;» anche nell'exit dei loop.

2. Cose da controllare durante l'analisi semantica statica

- Controllo tipi
- durante gli assegnamenti (e dichiarazioni) il numeri di L-expr e di R-expr deve essere lo stesso (Si può fare anche nel parser)
- Il numero di argomenti della funzione deve essere giusto
- Il numero di L-expr e il numero di tipi di ritorno nelle espressioni «x1, x2, .. = fun(p1, p2, ..)» deve essere uguale
- sostituire le costanti (da fare nel TAC?)
- nelle call delle routine annotare la lista dei parametri coi tipi, modalità e nomi.

3. Definizione del linguaggio

Il linguaggio definito per questo progetto è un linguaggio di programmazione imperativo con scoping statico. La sua grammatica si basa sulla sintassi concreta del linguaggio Blue.

Poiché Blue è un linguaggio orientato agli oggetti, per garantire le funzionalità richieste dalla consegna, come per esempio il dereferenziamento, sono stati introdotti alcuni elementi. Tuttavia, non sono presenti né classi né interfacce.

3.a. Tipi

Il linguaggio definito prevede i seguenti tipi primitivi: `Boolean`, `Integer`, `Real`, `String` e `Char`. Sono inoltre supportati gli arrayche sono dichiarati utilizzando nel modo seguente: `Array< Type, Integer>`, dove `Type` rappresenta il tipo degli elementi e `Integer` indica la dimensione dell'array.

3.a.I. Type system

3.a.I.I. Expression

3.a.I.I.I. Literal

$$\frac{}{\text{env} \vdash \text{Literal } \{\text{val} = \text{LitInt } \{\text{intVal} = e\}\} : \text{INT}} \tag{1}$$

$$\frac{}{\text{env} \vdash \text{Literal } \{\text{val} = \text{LitChar } \{\text{charVal} = e\}\} : \text{CHAR}} \tag{2}$$

$$\frac{}{\text{env} \vdash \text{Literal } \{\text{val} = \text{LitString } \{\text{stringVal} = e\}\} : \text{STRING}} \tag{3}$$

$$\frac{}{\text{env} \vdash \text{Literal } \{\text{val} = \text{LitBool } \{\text{boolVal} = e\}\} : \text{BOOL}} \tag{4}$$

$$\frac{}{\text{env} \vdash \text{Literal } \{\text{val} = \text{LitDouble } \{\text{doubleVal} = e\}\} : \text{REAL}} \tag{5}$$

3.a.I.I.II. Ident

$$\frac{}{\text{env} \vdash \text{ExpIdent } \{\text{name} = \text{id}\} : \text{env}(\text{id})} \tag{6}$$

3.b. Variabili

Le variabili sono dichiarate con la keyword `var` e possono essere di tipo primitivo o di tipo array. Non è consentito dichiarare variabili senza assegnamento.

3.c. Costanti

Le costanti sono dichiarate con la keyword `const` e possono essere di tipo primitivo (non devono allocare spazio in memoria). Anche in questo caso non è consentito dichiarare costanti senza assegnamento.

3.d. Loop

Per il nostro linguaggio sono previste le forme di iterazione non determinata `while-do` e `do-while`. Entrambe le strutture sono naturalmente implementate dall'elemento `loop-exit on` della sintassi concreta di Blue.

3.e. Condizionali

Il linguaggio immplementa le strutture condizionali semplici `if-then` e `if-then-else`, che permettono di eseguire blocchi di codice in base a condizioni specifiche.

3.f. Routine

Le routine comprendono sia funzioni che procedure. Una funzione priva di parametri in ingresso può essere invocata in entrambe le forme `function()` o `function`. Rispetto alla sintassi concreta del linguaggio Blue, sono rimosse pre e post-condizioni; inoltre i parametri di ritorno non vengono nominati esplicitamente nella dichiarazione: si specificano soltanto i tipi restituiti mediante la notazione `-> (<Type>)`. Per gestire il ritorno esplicito dei valori, è stato introdotto il costrutto `return`. Quando si dichiara una funzione nell'ambiente più esterno, è necessario utilizzare la keyword `routines`, in modo simile a quanto avviene in Blue per le dichiarazioni delle routine nelle classi.

Quando si invoca una funzione è possibile specificare la modalità di passaggio dei parametri in ingresso (`val`, `valres` e `ref`). Infine, poiché il linguaggio non prevede il concetto di classi, le funzioni non possono essere invocate con la notazione a punto, come `.function()`.

3.g. Commenti

I commenti supportati sono i commenti a riga singola, che iniziano con il simbolo `--`, e i commenti a blocco, che iniziano con `/*` e terminano con `*/`.

4. TODO

4.a. separatori

L'introduzione della dereferenziazione (deref) ha portato alla necessità di separatori per distinguere chiaramente tra le diverse parti delle espressioni. Infatti senza i separatori il parser LALR(1) non sarebbe in grado di distinguere un deref da una moltiplicazione, oppure una chiamata di funzione da un deref

```
Blue
```

5. TODO:

- [ ] disambiguare funzioni con lo stesso nome nel tac